

**Atal Bihari Vajpayee Indian Institute of  
Information Technology and Management, Gwalior**  
Department of Computer Science and Engineering

M.TECH THESIS — INTERIM PROGRESS REPORT

**Quantifying SZZ-Induced Label Noise in  
Just-In-Time Software Defect Prediction  
under Verification-Latency-Aware  
Online Evaluation**

Progress Report: Phase 1 and Phase 2

**Submitted by**

Puneet Deshwani

Roll No. 2025MCS013

M.Tech (CSE)

**Under the supervision of**

Dr. Santosh Singh Rathore

Department of CSE

ABV-IIITM Gwalior

September 2026

# Abstract

Just-in-time software defect prediction (JIT-SDP) flags risky commits at the moment they are made. Almost every JIT-SDP dataset takes its labels from the SZZ algorithm, which infers defect-introducing commits by tracing the lines changed in a bug fix back through version history. That inference fails when bug-fixing commits are tangled with unrelated edits, and because models are normally trained *and* evaluated on SZZ labels, the resulting errors are invisible to standard evaluation.

This thesis measures what SZZ-induced label noise costs a defect predictor under realistic deployment conditions. This report covers the first two of its four phases, on 27,319 commits from 21 Apache Java projects (JIT-Defects4J), using an independently constructed reference label set that is held out and used only for scoring.

**Phase 1** compares six SZZ variants with the reference labels. No variant exceeds 27.2% precision; the noise is class-conditional rather than symmetric (false-alarm rates 6.7–26.3%, miss rates 35.9–73.3%); and the variants agree with each other (Cohen’s  $\kappa$  up to 0.933) far more than with the reference ( $\kappa$  0.158–0.202). **Phase 2** trains three models on seven label sources under four evaluation regimes (16,380 runs, paired project-level tests with Holm correction). Random  $k$ -fold cross-validation inflates a high-capacity model by 0.137 MCC, and scoring it against its own training labels adds a further 0.230. Under streaming evaluation with verification latency, training on the reference labels beats every SZZ variant by 0.034–0.064 MCC. A result of 0.41 MCC under the common protocol falls to 0.10 under a realistic one. The remaining two phases will identify the mechanism behind the damage and test a noise-aware online learner.

# Contents

|                                                     |           |
|-----------------------------------------------------|-----------|
| <b>Abstract</b>                                     | <b>i</b>  |
| <b>1 Introduction</b>                               | <b>1</b>  |
| 1.1 Background and motivation . . . . .             | 1         |
| 1.2 Problem statement . . . . .                     | 1         |
| 1.3 Research questions . . . . .                    | 2         |
| 1.4 Objectives . . . . .                            | 2         |
| <b>2 Literature Review</b>                          | <b>2</b>  |
| 2.1 Just-in-time defect prediction . . . . .        | 2         |
| 2.2 The SZZ algorithm and its variants . . . . .    | 3         |
| 2.3 Evaluation methodology . . . . .                | 3         |
| 2.4 Online learning for JIT-SDP . . . . .           | 4         |
| 2.5 Model complexity and simple baselines . . . . . | 4         |
| 2.6 Learning with noisy labels . . . . .            | 4         |
| 2.7 Research gap . . . . .                          | 4         |
| <b>3 Dataset</b>                                    | <b>5</b>  |
| 3.1 The JIT-Defects4J corpus . . . . .              | 5         |
| 3.2 The reference labels . . . . .                  | 5         |
| 3.3 Features and preprocessing . . . . .            | 6         |
| 3.4 Verification latency . . . . .                  | 6         |
| <b>4 Methodology</b>                                | <b>7</b>  |
| 4.1 Overall research design . . . . .               | 7         |
| 4.2 Phase 1: measuring label quality . . . . .      | 7         |
| 4.3 Phase 2: measuring downstream impact . . . . .  | 7         |
| 4.4 Metrics and statistical protocol . . . . .      | 8         |
| <b>5 Results</b>                                    | <b>9</b>  |
| 5.1 Phase 1: SZZ label quality (RQ1) . . . . .      | 9         |
| 5.2 Phase 2: downstream impact (RQ2) . . . . .      | 11        |
| <b>6 Conclusion and Future Work</b>                 | <b>14</b> |
| 6.1 Summary of findings . . . . .                   | 14        |
| 6.2 Threats to validity . . . . .                   | 15        |
| 6.3 Planned work . . . . .                          | 15        |
| <b>References</b>                                   | <b>16</b> |

# 1 Introduction

## 1.1 Background and motivation

Software defects become more expensive the longer they survive. *Defect prediction* directs limited review and testing effort towards the code most likely to be faulty, and *just-in-time software defect prediction* (JIT-SDP) makes that prediction for each commit as it arrives, so that a risky change can be reviewed while its author still remembers it [1]. The setting has three defining properties: commits arrive as a **stream**, so evaluation must respect time; defect-introducing commits are **rare** (8.54% in our corpus); and **a commit’s label is unknown when it arrives** — it becomes known only once the defect is found and fixed.

Projects record which commits *fix* defects, not which introduced them. The SZZ algorithm [2] bridges this gap by applying `git blame` to the lines changed in a bug fix and labelling the commits that last touched those lines as defect-introducing. Its premise — that the lines a fix changes are the lines that were wrong — often fails, because fixes are *tangled* with refactoring, renaming and formatting: Herbold et al. [3] found that only 17–32% of the changes in bug-fixing commits address the bug itself. Since JIT-SDP models are usually both trained and evaluated on SZZ labels, such errors cannot be seen from inside the evaluation.

Evaluation adds a second problem. Random  $k$ -fold cross-validation, still common, lets a model train on future commits [4]. Even a chronological split assumes that training labels are available immediately; in our corpus the median delay between a commit and the fix that reveals its defect is 113 days. Cabral and Minku [5] called this *verification latency* and proposed ORB, the online learner used in this work.

## 1.2 Problem statement

*JIT-SDP models are trained and evaluated almost exclusively on labels produced by SZZ, using protocols that ignore commit order and verification latency. The field therefore has no measurement of what SZZ-induced label noise costs a defect predictor under realistic deployment conditions, and no way to separate a model’s ability to find defects from its ability to reproduce the heuristic that labelled its training data.*

Addressing this requires separating four things: (i) the noise in the labels; (ii) the inflation contributed by the evaluation protocol; (iii) the mechanism by which noise damages an online learner; and (iv) whether a learner can be made robust to it. **This report covers (i) and (ii)**, which are Phases 1 and 2 of the thesis; (iii) and (iv) are planned work (Section 6.3).

### 1.3 Research questions

- RQ1** How much do SZZ variants disagree with one another and with an independently constructed reference, and what is the direction of each variant’s labelling bias? *(Phase 1, this report.)*
- RQ2** How does the choice of SZZ label source shift measured JIT-SDP performance across naive, chronological and prequential (streaming, with latency) evaluation regimes? *(Phase 2, this report.)*
- RQ3** Through what mechanism does label noise degrade an online learner, and which of SZZ’s two error types — false positives or false negatives — does the damage? *(Phase 3, planned.)*
- RQ4** Can an online learner recover performance lost to SZZ-origin noise without clean labels, and without losing performance when labels are clean? *(Phase 4, planned.)*

### 1.4 Objectives

- O1.** Characterise the label noise of six SZZ variants against an independent reference, as class-conditional flip rates. *(Completed.)*
- O2.** Quantify how much the evaluation protocol — temporal leakage and scoring against the training labels — inflates measured performance, and separate this from the effect of the label source. *(Completed.)*
- O3.** Measure the cost of SZZ labels for an online learner evaluated as a stream with realistic verification latency. *(Completed.)*
- O4.** Identify which SZZ error type causes the damage, and by what mechanism. *(Planned, Phase 3.)*
- O5.** Design and evaluate a noise-aware online learner under a pre-registered protocol. *(Planned, Phase 4.)*

The report is organised as follows: Section 2 reviews related work, Section 3 describes the dataset, Section 4 the methodology, Section 5 the results of Phases 1 and 2, and Section 6 concludes and outlines the remaining work.

## 2 Literature Review

### 2.1 Just-in-time defect prediction

Kamei et al. [1] established the modern form of JIT-SDP, defining 14 change-level metrics in five families (Table 3) and reporting about 68% accuracy and 64% recall

on defect-inducing changes. A systematic survey by Zhao et al. [6] catalogues 67 JIT-SDP studies built largely on this formulation. Because defect-introducing commits are a small minority, accuracy is uninformative — predicting “clean” everywhere scores 91.5% on our corpus — and oversampling of the minority class is standard practice.

## 2.2 The SZZ algorithm and its variants

SZZ [2] extracts the lines modified by a bug fix, applies `git blame` to find the last commit that touched each of them, and labels those commits defect-introducing. Each of its two premises has a known failure mode: *tangled commits*, in which unrelated lines of a fix are blamed as well [3], and *blame attribution error*, in which the last commit to touch a line is not the one that made it wrong. Later variants (Table 1) add filters to remove the original algorithm’s over-collection.

**Table 1:** SZZ variants studied in this work.

| Variant     | Mechanism                                                                    |
|-------------|------------------------------------------------------------------------------|
| B-SZZ [2]   | The original: blame every line modified by the fix                           |
| AG-SZZ [7]  | Annotation graphs; filters cosmetic changes such as blank lines and comments |
| MA-SZZ [8]  | Also excludes meta-changes such as branch merges and property changes        |
| R-SZZ [9]   | Keeps only the most recent candidate commit                                  |
| L-SZZ [9]   | Keeps only the candidate with the most changed lines                         |
| RA-SZZ [10] | Refactoring-aware; removes candidates caused by refactorings (Java only)     |

Evaluating SZZ needs a reference that does not come from SZZ. Rosa et al. [11, 12] built a *developer-informed oracle* from commit messages that explicitly name the bug-introducing commit and found R-SZZ best (precision  $\approx 66\%$ ). That oracle, however, contains no clean commits, so it cannot train a JIT-SDP model, and it scores SZZ only on fixes already known to have an inducing commit, which is a different question from labelling every commit in a project’s history.

## 2.3 Evaluation methodology

Falessi et al. [4] showed that validation schemes which do not preserve the time order of data give misleading estimates for defect classifiers, yet random  $k$ -fold remains common in JIT-SDP. For streams the standard protocol is *prequential* (test-then-train) evaluation; Gama et al. [13] show that prequential *error* with fading factors converges to the Bayes error, although this guarantee does not extend directly to non-decomposable metrics such as MCC. Cabral and Minku [5] identified *verification latency* as a first-class concern: a clean commit is only ever presumed clean, while a defective one must be discovered. Their later work [14] reports discovery delays from one day to over eleven years, with medians typically at or below 90 days, which is the basis for our 90-day window.

## 2.4 Online learning for JIT-SDP

Online Bagging [15] presents each arriving example to each ensemble member  $k \sim \text{Poisson}(\lambda)$  times. Oversampling Online Bagging [16] sets  $\lambda$  from the running class ratio so that minority examples are repeated more often, and **ORB** [5] further boosts the resampling rate of the class the model is currently biased against. In all three,  $\lambda$  depends on the *number* of defect labels and not on their correctness, so a false positive is amplified exactly as much as a true one. This observation motivates Phase 3.

## 2.5 Model complexity and simple baselines

Deep models such as DeepJIT [17] and CC2Vec [18] reported large gains, but Zeng et al. [19] found that **LApredict**, a logistic regression on the added-lines feature alone, outperformed both, and **JITLine** [20], a simpler feature-based model, was more accurate and much faster than either. We use LApredict and JITLine because they sit at opposite ends of model capacity.

## 2.6 Learning with noisy labels

Natarajan et al. [21] study class-conditional label noise — the noise model Phase 1 measures — and derive an unbiased loss that becomes unstable as  $\rho_0 + \rho_1 \rightarrow 1$ . Co-teaching [22] trains two networks that exchange small-loss examples, and Confident Learning [23] estimates the joint distribution of noisy and true labels to prune suspect examples. All three assume a fixed dataset, repeated passes and labels that are wrong but stable, whereas a JIT-SDP stream gives a single pass and labels that arrive late and may later be corrected.

## 2.7 Research gap

SZZ evaluation research measures label noise but stops at the labels. Online JIT-SDP research handles latency and imbalance but treats label noise as a nuisance rather than a measured quantity. Noisy-label learning assumes a noise model instead of estimating one, in a batch setting. **No existing work measures the downstream cost of SZZ-origin noise — asymmetric, variant-dependent, with known flip rates — under latency-aware online evaluation.** What makes this measurable here is an independently constructed reference label set, held out from training and used only for scoring.

### 3 Dataset

#### 3.1 The JIT-Defects4J corpus

All experiments use the **JIT-Defects4J** dataset [24], an extension of the LLTC4J tangling dataset [3]. It contains **27,319 commits from 21 Apache Java projects**, of which **2,332 (8.54%)** are defect-introducing. Projects range from 544 to 4,026 commits, 19 to 335 defects (defect rate 1.8–19.3%) and 5.9 to 17.9 years of history (Table 2). The corpus was adopted whole, as published, so that project selection could not be tuned to the results.

**Table 2:** Per-project statistics. Defects are reference defect-introducing commits; rate is defects as a percentage of commits.

| Project               | Commits | Defects | Rate % | Project           | Commits       | Defects      | Rate %     |
|-----------------------|---------|---------|--------|-------------------|---------------|--------------|------------|
| ant-ivy               | 1,771   | 332     | 18.8   | commons-lang      | 2,969         | 146          | 4.9        |
| commons-bcel          | 825     | 60      | 7.3    | commons-math      | 4,026         | 335          | 8.3        |
| commons-beanutils     | 611     | 37      | 6.1    | commons-net       | 1,121         | 117          | 10.4       |
| commons-codec         | 761     | 36      | 4.7    | commons-scxml     | 544           | 47           | 8.6        |
| commons-collections   | 1,823   | 50      | 2.7    | commons-validator | 598           | 36           | 6.0        |
| commons-compress      | 1,630   | 178     | 10.9   | commons-vfs       | 1,110         | 114          | 10.3       |
| commons-configuration | 1,838   | 155     | 8.4    | giraph            | 844           | 163          | 19.3       |
| commons-dbcp          | 1,037   | 58      | 5.6    | gora              | 553           | 39           | 7.0        |
| commons-digester      | 1,079   | 19      | 1.8    | opennlp           | 1,086         | 91           | 8.4        |
| commons-io            | 1,142   | 73      | 6.4    | parquet-mr        | 1,120         | 158          | 14.1       |
| commons-jcs           | 831     | 88      | 10.6   | <b>Total</b>      | <b>27,319</b> | <b>2,332</b> | <b>8.5</b> |

#### 3.2 The reference labels

The reference labels come from JIT-Defects4J and were built in two stages. Lines inside bug-fixing commits were annotated manually and kept only where at least three annotators agreed that the line fixes the bug; `git blame` was then applied to those verified lines to find the introducing commits, and all other commits are clean. We confirmed this provenance against the dataset’s replication package, matching the published defect counts project by project.

The reference is therefore **git blame seeded with human-verified fix lines**, not manually verified ground truth. Three consequences follow. (i) Comparing an SZZ variant with it isolates the effect of *tangling*, since both sides use the same blame step and differ only in which fix lines seed it. (ii) Blame attribution error is present on both sides and cancels, so **all measured noise is a lower bound**. (iii) Every reference positive is reachable by blame, so no measured false negative can be blamed on blame being unable to trace a defect.

### 3.3 Features and preprocessing

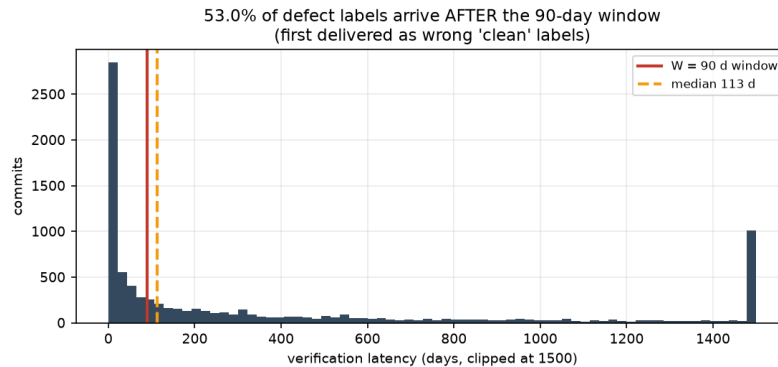
Each commit is described by the 14 Kamei change-level metrics (Table 3), used as published rather than recomputed so that feature extraction cannot differ between label sources. Commits are de-duplicated on (project, commit hash) and sorted by author timestamp, which fixes the stream order for every temporal regime. The raw inputs (a 101 MB archive and 830 MB of cloned repositories) were reduced to two small derived files, so the whole study runs from committed inputs.

**Table 3:** The Kamei change-level features [1].

| Family     | Features            | Intuition                                                                |
|------------|---------------------|--------------------------------------------------------------------------|
| Diffusion  | NS, ND, NF, Entropy | Changes spread over many subsystems and files are harder to reason about |
| Size       | LA, LD, LT          | Larger changes give more opportunity for error                           |
| Purpose    | FIX                 | Fixing changes are themselves defect-prone                               |
| History    | NDEV, AGE, NUC      | Code changed often, recently, by many developers                         |
| Experience | EXP, REXP, SEXP     | The author’s familiarity with the changed code                           |

### 3.4 Verification latency

To model when a label would really become known, we extracted the author date of every bug-fixing commit from the project repositories; a commit’s label arrives on the date of the fix that blamed it. As Figure 1 shows, the median latency is 113 days and **53% of defect labels arrive after a 90-day window**. The published reference labels carry no fix-to-inducing links, so their arrival times were reconstructed from the SZZ mappings (coverage 67.8%); the reference is therefore SZZ-derived in timing, which is the main validity threat for the streaming results (Section 6.2).



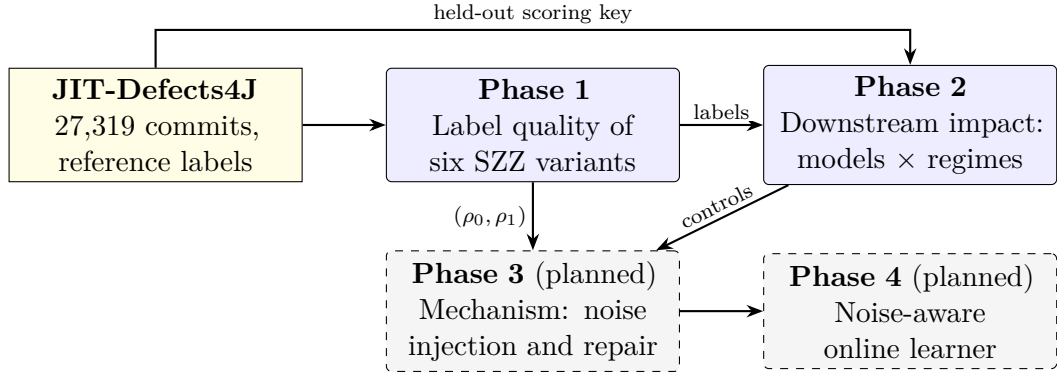
**Figure 1:** Verification latency (days from a commit to the fix that blames it) over all commits linked to a bug fix. Labels to the right of the 90-day window (red line) first reach the learner as incorrect “clean” labels.

Larger corpora such as ApacheJIT [25] and Defectors [26] were not used because their labels are SZZ-derived and so cannot serve as a reference for measuring SZZ noise.

## 4 Methodology

### 4.1 Overall research design

The thesis has four phases linked by data dependencies (Figure 2). Phase 1 measures label noise and produces per-commit labels and flip rates; Phase 2 uses those labels to measure downstream impact; the planned Phases 3 and 4 will use the flip rates to calibrate noise experiments and then design a mitigation. Every claim is a paired comparison at the project level, and every number in this report is regenerated from committed inputs by a named script.



**Figure 2:** Phase pipeline. Solid boxes are reported here; dashed boxes are planned work.

### 4.2 Phase 1: measuring label quality

Six SZZ variants (Table 1) were run with **PySZZ v2** [12], the reference implementation released with the developer-informed-oracle study, on the same set of bug-fixing commits. Each variant’s output was aligned to the reference over an **identical 27,319-commit denominator**, since a variant scored on a partial denominator can look more precise simply by being evaluated on fewer commits. For each variant we compute precision, recall, F1, MCC and Cohen’s  $\kappa$  against the reference, and pairwise  $\kappa$  between variants. Each variant is also described as a class-conditional noisy channel between the reference label  $y$  and the variant label  $\tilde{y}$ :

$$\rho_0 = P(\tilde{y} = 1 \mid y = 0) \text{ (false-alarm rate)}, \quad \rho_1 = P(\tilde{y} = 0 \mid y = 1) \text{ (miss rate)}. \quad (1)$$

The six  $(\rho_0, \rho_1)$  pairs are saved as a reusable artefact for calibrating the Phase 3 noise experiments.

### 4.3 Phase 2: measuring downstream impact

Phase 2 is a four-way factorial experiment (Table 4) over 21 projects and 10 random seeds, giving **16,380 evaluation records**.

**Table 4:** Phase 2 experimental design.

| Factor                | Levels                                                                                                                                                                                        |
|-----------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Model (3)             | LApredict (logistic regression on lines added); JITLine (100-tree random forest with SMOTE and threshold moving); ORB (20-member online logistic ensemble with adaptive Poisson oversampling) |
| Training labels (7)   | Reference labels; B-, AG-, MA-, L-, R- and RA-SZZ                                                                                                                                             |
| Evaluation regime (4) | Naive 10-fold CV; chronological 50/50 split; chronological-online (ORB trained on the first half, then frozen); prequential with verification latency                                         |
| Scoring (2)           | Oracle-scored (against the held-out reference labels); self-scored (against the training labels)                                                                                              |

**Scoring convention.** This is the core of the design. A *self-scored* model is evaluated against the same SZZ labels it was trained on, which is the usual practice in the literature. An *oracle-scored* model is trained on SZZ labels but evaluated against the reference labels, which are never used for training. Self-scoring measures how faithfully a model reproduces a heuristic; oracle-scoring measures how well it finds real defects. The gap between the two is measurable only because the reference is held out.

**Latency protocol.** In the prequential regime each commit is predicted on arrival at time  $t$  and scored at once against the reference. Its training label is released with a window  $W = 90$  days: a clean label at  $t + W$ ; a defective label at the fix time if the fix falls within the window; and if the fix comes later, *first as a clean label* at  $t + W$  and then corrected at the fix time. Defective commits with no linked fix are delivered as clean and never corrected. A learner under latency is therefore actively trained on labels it will later discover were wrong, and these are exactly the minority-class examples it most needs.

#### 4.4 Metrics and statistical protocol

The primary metric is the **Matthews correlation coefficient** [27], which uses all four cells of the confusion matrix and is zero for any uninformed predictor:

$$\text{MCC} = \frac{TP \cdot TN - FP \cdot FN}{\sqrt{(TP + FP)(TP + FN)(TN + FP)(TN + FN)}}. \quad (2)$$

G-mean is reported alongside, and Cohen’s  $\kappa$  [28] measures agreement between labelling procedures. In the streaming regime MCC is a trajectory rather than one number. We use its *time-averaged* value as the primary summary and a *terminal* value from a fading confusion matrix (factor 0.99) as a secondary one. Neither is canonical, because MCC is not decomposable; the time-averaged value is preferred because its project-level variance is about half as large (0.051 against 0.092).

Every test is **paired at the project level** ( $n = 21$ , seeds averaged before pair-

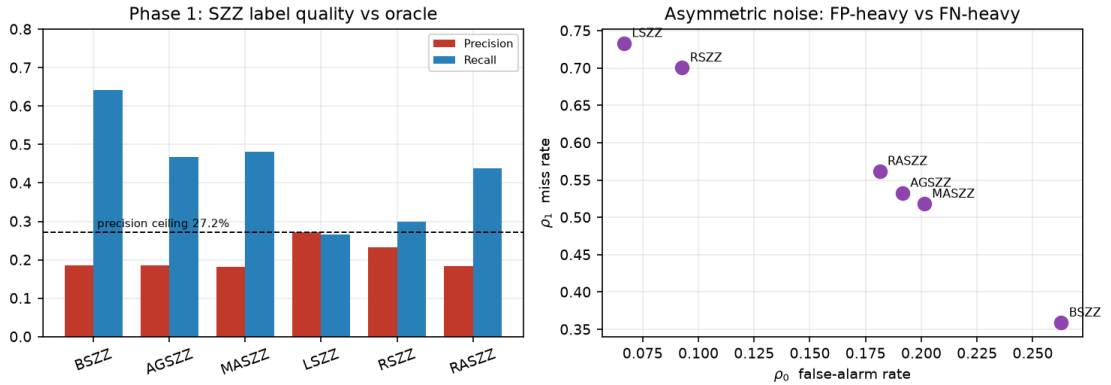
ing), since pooling the 16,380 records would be pseudo-replication. Each comparison reports the Wilcoxon signed-rank test [29], the matched-pairs rank-biserial correlation (+1 means every project moved in the same direction), the Hodges–Lehmann (HL) estimate [30] and a bootstrap 95% confidence interval for it. Phase 2 runs 74 tests, corrected with Holm’s method [31] both within each test family and **globally across all 74**; headline claims rest on the global correction. Phases 1 and 2 are exploratory and were not pre-registered.

## 5 Results

### 5.1 Phase 1: SZZ label quality (RQ1)

**Table 5:** Quality of each SZZ variant against the reference labels (27,319 commits, 2,332 reference positives).

| Variant | Prec.        | Recall       | F1           | $\rho_0$     | $\rho_1$     | MCC          | $\kappa$     | TP    | FP    | FN    |
|---------|--------------|--------------|--------------|--------------|--------------|--------------|--------------|-------|-------|-------|
| B-SZZ   | 0.186        | <b>0.641</b> | <b>0.288</b> | 0.263        | <b>0.359</b> | <b>0.232</b> | 0.179        | 1,495 | 6,565 | 837   |
| AG-SZZ  | 0.186        | 0.467        | 0.266        | 0.192        | 0.533        | 0.188        | 0.163        | 1,090 | 4,786 | 1,242 |
| MA-SZZ  | 0.183        | 0.482        | 0.265        | 0.201        | 0.518        | 0.188        | 0.161        | 1,124 | 5,030 | 1,208 |
| L-SZZ   | <b>0.272</b> | 0.267        | 0.269        | <b>0.067</b> | 0.733        | 0.202        | <b>0.202</b> | 622   | 1,665 | 1,710 |
| R-SZZ   | 0.232        | 0.300        | 0.262        | 0.093        | 0.700        | 0.185        | 0.183        | 699   | 2,316 | 1,633 |
| RA-SZZ  | 0.184        | 0.438        | 0.259        | 0.181        | 0.562        | 0.178        | 0.158        | 1,022 | 4,531 | 1,310 |



**Figure 3:** Left: precision and recall per variant, with the 27.2% precision ceiling dashed. Right: false-alarm rate  $\rho_0$  against miss rate  $\rho_1$ ; the variants spread along a trade-off rather than clustering.

#### 5.1.1 Per-variant quality

Table 5 and Figure 3 show three findings. **(1) A precision ceiling of 27.2%:** between 72.8% and 81.5% of the commits any variant flags are not defect-introducing according to the reference. **(2) Asymmetric noise that differs by variant:** B-SZZ is false-positive-heavy, flagging 8,060 commits to catch 1,495 real ones ( $\rho_0 = 0.263$ ), while L-SZZ and R-SZZ are the mirror image, with few false alarms but 73.3% and 70.0% of real

defects missed. Modelling SZZ noise as symmetric random flips, the usual assumption in noisy-label learning, therefore misrepresents it. **(3) No variant dominates:** MCC ranks B-SZZ first and RA-SZZ, the most refined variant, last;  $\kappa$  ranks L-SZZ first; F1 compresses all six into 0.259–0.288.

Rates and volumes also differ: B-SZZ’s  $\rho_0 = 0.263$  yields 6,565 false positives, while L-SZZ’s larger  $\rho_1 = 0.733$  yields only 1,710 false negatives, because the positive class is small. Per-project spread is large (B-SZZ’s recall ranges from 0.220 to 0.919), so all later tests are paired by project.

### 5.1.2 Agreement between variants

**Table 6:** Pairwise Cohen’s  $\kappa$  between variants. Each variant’s agreement with the reference is only 0.158–0.202.

|        | B-SZZ | AG-SZZ       | MA-SZZ | L-SZZ | R-SZZ |
|--------|-------|--------------|--------|-------|-------|
| AG-SZZ | 0.690 |              |        |       |       |
| MA-SZZ | 0.695 | <b>0.933</b> |        |       |       |
| L-SZZ  | 0.330 | 0.467        | 0.477  |       |       |
| R-SZZ  | 0.427 | 0.587        | 0.596  | 0.636 |       |
| RA-SZZ | 0.630 | 0.859        | 0.883  | 0.477 | 0.566 |

The annotation-graph family (AG, MA, RA) agrees internally at  $\kappa$  0.859–0.933 (“almost perfect”), yet each variant agrees with the reference at only 0.158–0.202 (“slight”; Table 6). **The variants agree with each other four to six times more strongly than with the reference:** they share a blame step and therefore share failure modes. High agreement between SZZ tools is sometimes read as evidence of correctness; on this corpus it is evidence of shared error.

### 5.1.3 Where the false negatives come from

Splitting each variant’s false negatives against B-SZZ reveals two mechanisms of similar size. *Seed-line divergence* accounts for 47–63%: SZZ and the reference blame different lines of the same fix and land on different commits. *Self-inflicted filtering* accounts for 37–53%: these are defects that B-SZZ found correctly and the variant’s own refinement then discarded. For the two most conservative variants this is the larger share — R-SZZ and L-SZZ discard 55.2% and 60.4% of the reference defects that B-SZZ had found. The refinements do cut false alarms ( $\rho_0$  falls from 0.263 to 0.067), but more than half of what the most aggressive filters remove is correct.

### 5.1.4 Answer to RQ1

SZZ variants disagree with the reference far more than with each other ( $\kappa$  0.158–0.202 against up to 0.933), and their bias is bifurcated in a way that follows from their design: B-SZZ over-flags because it blames every modified line, while R-SZZ and L-SZZ under-flag because they reduce the candidate set to a single commit. **Refinement**

has moved error from the false-positive column to the false-negative column rather than reducing it. Our precision values (0.18–0.27) are lower than those of Rosa et al. because the denominators differ: they ask which earlier commit introduced a *known* defect, whereas we ask whether each of 27,319 commits, most of them clean, is defect-introducing.

## 5.2 Phase 2: downstream impact (RQ2)

### 5.2.1 Overall results

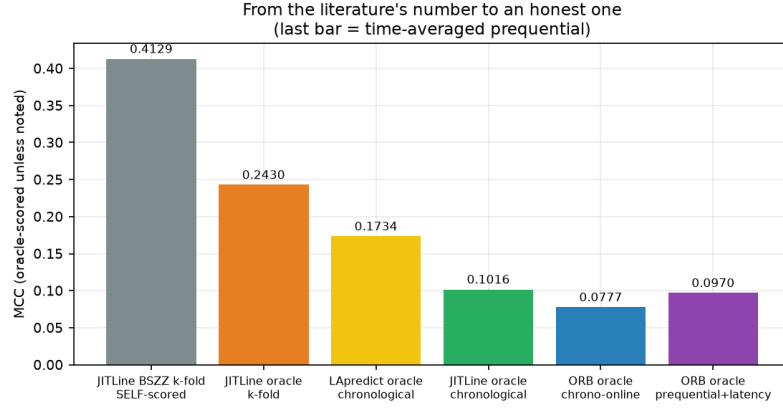
Table 7 gives the batch results. Under a chronological split, **LApredict** — a one-feature logistic regression — is the best model (0.173 against JITLine’s 0.102 on reference labels): JITLine’s advantage exists only under  $k$ -fold. The online model ORB, trained on reference labels, reaches 0.078 in the chronological-online regime and 0.097 in the prequential regime with latency.

**Table 7:** Batch models: mean MCC over 21 projects  $\times$  10 seeds. For reference labels the two scoring conventions coincide.

| Model     | Labels    | Naive $k$ -fold |               | Chronological |        |
|-----------|-----------|-----------------|---------------|---------------|--------|
|           |           | Oracle          | Self          | Oracle        | Self   |
| JITLine   | Reference | <b>0.2430</b>   | –             | 0.1016        | –      |
|           | B-SZZ     | 0.1751          | <b>0.4129</b> | <b>0.1324</b> | 0.2232 |
|           | AG-SZZ    | 0.1164          | 0.3037        | 0.0904        | 0.1351 |
|           | MA-SZZ    | 0.1272          | 0.3295        | 0.0786        | 0.1395 |
|           | L-SZZ     | 0.1391          | 0.2434        | 0.0804        | 0.1482 |
|           | R-SZZ     | 0.1131          | 0.1801        | 0.0626        | 0.0920 |
|           | RA-SZZ    | 0.1164          | 0.3070        | 0.0857        | 0.1402 |
| LApredict | Reference | 0.2058          | –             | 0.1734        | –      |
|           | B-SZZ     | 0.1941          | 0.3510        | 0.1634        | 0.2918 |
|           | AG-SZZ    | 0.2003          | 0.2472        | 0.1685        | 0.1975 |
|           | MA-SZZ    | 0.1995          | 0.2626        | 0.1664        | 0.2003 |
|           | L-SZZ     | 0.2078          | 0.2679        | 0.1706        | 0.2174 |
|           | R-SZZ     | 0.2016          | 0.1779        | 0.1731        | 0.1666 |
|           | RA-SZZ    | 0.2000          | 0.2603        | 0.1679        | 0.2143 |

### 5.2.2 The inflation ladder

Figure 4 measures the same task under progressively more realistic conditions. JIT-Line trained on B-SZZ, evaluated by  $k$ -fold and scored against its own labels — the configuration common in the literature — reaches **0.413 MCC**. Scoring it against the reference gives 0.175; training on reference labels with a chronological split gives 0.102; and ORB in a prequential stream with latency gives **0.097**. The ladder is descriptive, not causal: its last step changes several things at once (Section 5.2.6).



**Figure 4:** The inflation ladder, from the literature’s usual configuration (left) to a realistic one (right). All bars except the first are oracle-scored.

### 5.2.3 Result 1: temporal leakage inflates in proportion to capacity

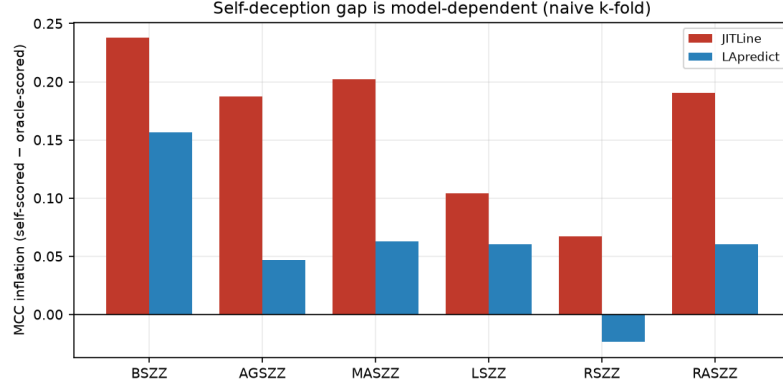
Moving from random  $k$ -fold to a chronological split, JITLine on reference labels loses **0.137 MCC** (HL  $+0.137$ , 95% CI  $[+0.105, +0.172]$ , rank-biserial  $+1.000$ , global Holm  $p = 7.1 \times 10^{-5}$ ): all 21 projects without exception. LAPredict loses only 0.034, which does not survive correction. A one-feature model has little capacity to exploit leakage, while a 100-tree forest has a great deal, so the apparent superiority of high-capacity models over simple baselines is partly an artefact of the protocol that measures it.

### 5.2.4 Result 2: self-scoring inflates further, depending on the model

Scoring the same predictions against the training labels rather than the reference produces **the largest effect in the study**:  $+0.230$  MCC for JITLine on B-SZZ (CI  $[+0.189, +0.278]$ , global Holm  $p = 1.0 \times 10^{-4}$ ). A random forest trained on B-SZZ appears about two and a half times better than it is, because it is graded by the heuristic that taught it. Under JITLine the gap is significant for all six variants ( $+0.18$  to  $+0.23$  for the four most permissive ones); under LAPredict it is significant only for B-SZZ ( $+0.155$ ) and is negative for R-SZZ (Figure 5). This capacity dependence is what one would expect if a model fits the error structure of its labels, and we tested it directly: among the 8,060 commits B-SZZ flags, a random forest separates its false positives from its true positives with mean AUC 0.599, against 0.514 with permuted labels (HL  $+0.088$ , 17 of 21 projects,  $p = 0.0016$ ). SZZ’s errors are learnable, although only modestly.

### 5.2.5 Result 3: under realistic streaming, the label source matters

With ORB in a prequential stream with real latency, training on the reference labels beats training on every SZZ variant (Table 8, Figure 6): all six intervals exclude zero, and five of six comparisons survive global correction. The **B-SZZ row is the most informative**: since B-SZZ and the reference share the blame step and differ only in which fix lines seed it, this contrast is close to a controlled ablation of tangled commits. Knowing which lines of a fix actually fix the bug is worth  **$+0.041$  MCC** to a deployed

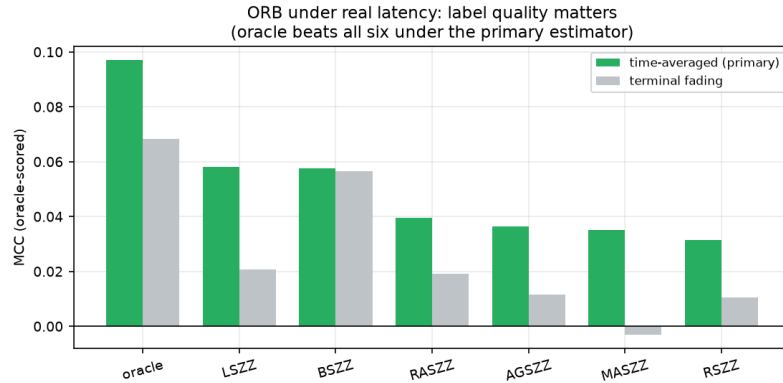


**Figure 5:** Self-scoring gap (self-scored minus oracle-scored MCC) under  $k$ -fold for JITLine (red) and LAPredict (blue). The gap is far larger for the high-capacity model.

**Table 8:** ORB, prequential evaluation with latency: reference-trained minus variant-trained MCC (time-averaged, oracle-scored).

| Reference vs | HL     | 95% CI           | Projects | Holm (family) | Holm (global) |
|--------------|--------|------------------|----------|---------------|---------------|
| R-SZZ        | +0.060 | [+0.042, +0.082] | 19/21    | 0.0001        | 0.0009        |
| MA-SZZ       | +0.064 | [+0.039, +0.085] | 18/21    | 0.0003        | 0.0042        |
| AG-SZZ       | +0.060 | [+0.029, +0.085] | 17/21    | 0.0009        | 0.0243        |
| RA-SZZ       | +0.058 | [+0.036, +0.081] | 18/21    | 0.0004        | 0.0064        |
| B-SZZ        | +0.041 | [+0.022, +0.052] | 18/21    | 0.0004        | 0.0064        |
| L-SZZ        | +0.034 | [+0.014, +0.055] | 16/21    | 0.0025        | 0.1241        |

online learner. The gap does not follow recall (correlation  $-0.18$ ): L-SZZ (recall 0.267) and B-SZZ (recall 0.641) have almost the same gap. The two estimators agree on every comparison except one — under the terminal estimator the B-SZZ row loses significance.



**Figure 6:** ORB under real verification latency: MCC by training label source, for the primary (time-averaged) and secondary (terminal) estimators.

### 5.2.6 Supplementary analyses

**Sizing the latency effect.** Read directly off the ladder, latency would account for about 9% of the batch-to-stream drop, but that contrast is not identified, because the learner, the label timing and the evaluation window all change together. A  $2 \times 2$

factorial that separates the learner (frozen or adaptive) from the labels (immediate or delayed) estimates a latency effect of about  $+0.023$  and an adaptivity effect of about  $-0.046$ , but **every contrast, including the interaction, has a confidence interval spanning zero**. With 21 projects, effects below roughly 0.04 MCC cannot be resolved, so this is reported as unresolved rather than null; a design that holds the learner fixed is needed (Phase 3).

**The JITLine anomaly.** Under the chronological split, JITLine trained on the noisier B-SZZ labels (0.132) beats JITLine trained on reference labels (0.102) in 15 of 21 projects. The effect is genuine rather than seed noise (97.7% of the between-project variance is real, and it survives four threshold-selection protocols), but none of the nine project characteristics tested — including defect count, B-SZZ precision and recall, and flag rate — predicts where it occurs. It is also batch-only: under streaming, the reference beats B-SZZ in 18 of 21 projects. We report it as a robust observation without a claimed mechanism.

### 5.2.7 Robustness

The label-source result was re-run over 20 settings of fading factor (0.90–0.999) and warm-up length (0–25%) for all six variants: in **120 of 120** comparisons the HL estimate favours the reference, the interval excludes zero and the result survives grid-wide Holm correction. Excluding the eight projects with fewer than 25 test-half defects moves no headline estimate by more than 0.002 MCC. One secondary result weakens: the self-scoring gap under the chronological regime falls from 0.098 to 0.075 and loses significance.

### 5.2.8 Answer to RQ2

The label source shifts measured performance significantly under every regime that permits a fair test: under realistic streaming, training on SZZ instead of the reference costs 0.034–0.064 MCC, between a third and two-thirds of the available signal (0.097). **The evaluation protocol shifts it far more.** Random  $k$ -fold inflates JITLine by 0.137 and self-scoring adds a further 0.230; together the protocol effects are about nine times the label-source effect, and both grow with model capacity. The two problems are linked, because self-scoring is possible only when the labels are heuristic.

## 6 Conclusion and Future Work

### 6.1 Summary of findings

- **SZZ labels are far noisier than usually assumed.** No variant exceeds 27.2% precision against the reference, and the noise is class-conditional: B-SZZ is false-positive-heavy, L-SZZ and R-SZZ false-negative-heavy.

- **Agreement between SZZ variants is not evidence of correctness:**  $\kappa$  up to 0.933 with one another, but only 0.158–0.202 with the reference.
- **Refinement relocates error rather than reducing it:** more than half of what the most aggressive filters remove is a correctly found defect.
- **The evaluation protocol outweighs the label choice:** temporal leakage (+0.137) and self-scoring (+0.230) together are about nine times the label-source effect (0.034–0.064), and both grow with model capacity.
- **Most reported performance is a measurement artefact:** 0.41 MCC under the common protocol is 0.10 under a realistic one. Better labels alone would not make JIT-SDP strong, but the field’s reported numbers overstate its current capability roughly four-fold.

## 6.2 Threats to validity

**Construct:** the reference is `git blame` seeded with verified fix lines rather than ground truth, so all measured noise is a lower bound; and its label arrival times are SZZ-derived (coverage 67.8%), which is the most serious threat to the streaming results.

**Internal:** the corpus is SZZ-shaped, since the dataset excludes changes that add no lines, so defects introduced purely by deletion cannot appear. **Conclusion:** with 21 paired projects, effects below about 0.04 MCC are not resolvable, and the analyses are exploratory, which is why headline claims rest on global Holm correction. **External:** one benchmark family of 21 Apache Java projects.

## 6.3 Planned work

**Phase 3 — mechanism (RQ3).** Phase 2 shows that label quality matters, but not whether the cost comes from the false positives SZZ adds or from the defects it misses; comparing label sources cannot settle this, because each source differs in many ways at once. Phase 3 will therefore intervene: it will inject noise calibrated to the Phase 1 flip rates into the reference labels, and separately repair real B-SZZ labels one error type at a time, keeping the stream, learner and latency model fixed. Phase 2 dictates three controls: a *matched-mass* condition, because B-SZZ has about eight false positives for every false negative; an *immediate-delivery* condition, so that repaired labels are not lost to latency; and a true *no-latency* arm in which every label arrives at once. ORB’s internal oversampling rate will also be recorded, to test whether its amplification of defect labels — driven by their number, not their correctness — is the route by which noise does damage.

**Phase 4 — mitigation (RQ4).** Guided by the Phase 3 findings, Phase 4 will design a noise-aware online learner that does not require clean labels. It will be

**pre-registered** — hypotheses, primary model, acceptance criteria and multiplicity correction committed before the first run — and its acceptance bar will include a *non-degradation* check, since a method that helps on noisy labels but hurts on clean ones cannot be adopted by a practitioner who does not know which case applies.

## References

- [1] Y. Kamei, E. Shihab, B. Adams, A. E. Hassan, A. Mockus, A. Sinha, and N. Ubayashi, “A large-scale empirical study of just-in-time quality assurance,” *IEEE Transactions on Software Engineering*, vol. 39, no. 6, pp. 757–773, 2013.
- [2] J. Śliwerski, T. Zimmermann, and A. Zeller, “When do changes induce fixes?” in *Proc. International Workshop on Mining Software Repositories (MSR)*, 2005, pp. 1–5.
- [3] S. Herbold, A. Trautsch, B. Ledel, *et al.*, “A fine-grained data set and analysis of tangling in bug fixing commits,” *Empirical Software Engineering*, 2022, doi: 10.1007/s10664-021-10083-5.
- [4] D. Falessi, J. Huang, L. Narayana, J. F. Thai, and B. Turhan, “On the need of preserving order of data when validating within-project defect classifiers,” *Empirical Software Engineering*, vol. 25, pp. 4805–4830, 2020.
- [5] G. G. Cabral, L. L. Minku, E. Shihab, and S. Mujahid, “Class imbalance evolution and verification latency in just-in-time software defect prediction,” in *Proc. 41st IEEE/ACM International Conference on Software Engineering (ICSE)*, 2019.
- [6] Y. Zhao, K. Damevski, and H. Chen, “A systematic survey of just-in-time software defect prediction,” *ACM Computing Surveys*, vol. 55, no. 10, pp. 1–35, 2023.
- [7] S. Kim, T. Zimmermann, K. Pan, and E. J. Whitehead, Jr., “Automatic identification of bug-introducing changes,” in *Proc. 21st IEEE/ACM International Conference on Automated Software Engineering (ASE)*, 2006, pp. 81–90.
- [8] D. A. da Costa, S. McIntosh, W. Shang, U. Kulesza, R. Coelho, and A. E. Hassan, “A framework for evaluating the results of the SZZ approach for identifying bug-introducing changes,” *IEEE Transactions on Software Engineering*, vol. 43, no. 7, pp. 641–657, 2017.
- [9] S. Davies, M. Roper, and M. Wood, “Comparing text-based and dependence-based approaches for determining the origins of bugs,” *Journal of Software: Evolution and Process*, vol. 26, no. 1, pp. 107–139, 2014.
- [10] E. C. Neto, D. A. da Costa, and U. Kulesza, “The impact of refactoring changes on the SZZ algorithm: An empirical study,” in *Proc. IEEE 25th International Conference on Software Analysis, Evolution and Reengineering (SANER)*, 2018, pp. 380–390.
- [11] G. Rosa, L. Pascarella, S. Scalabrino, R. Tufano, G. Bavota, M. Lanza, and R. Oliveto, “Evaluating SZZ implementations through a developer-informed oracle,” in *Proc. 43rd IEEE/ACM International Conference on Software Engineering (ICSE)*, 2021.
- [12] G. Rosa, L. Pascarella, S. Scalabrino, R. Tufano, G. Bavota, M. Lanza, and R. Oliveto, “A comprehensive evaluation of SZZ variants through a developer-informed oracle,” *Journal of Systems and Software*, vol. 202, p. 111729, 2023.
- [13] J. Gama, R. Sebastião, and P. P. Rodrigues, “On evaluating stream learning algorithms,” *Machine Learning*, vol. 90, no. 3, pp. 317–346, 2013.
- [14] G. G. Cabral and L. L. Minku, “Towards reliable online just-in-time software defect prediction,” *IEEE Transactions on Software Engineering*, 2023.
- [15] N. C. Oza and S. Russell, “Online bagging and boosting,” in *Proc. 8th International Workshop on Artificial Intelligence and Statistics (AISTATS)*, 2001, pp. 229–236.
- [16] S. Wang, L. L. Minku, and X. Yao, “Resampling-based ensemble methods for online class imbalance learning,” *IEEE Transactions on Knowledge and Data Engineering*, vol. 27, no. 5, pp. 1356–1368, 2015.
- [17] T. Hoang, H. K. Dam, Y. Kamei, D. Lo, and N. Ubayashi, “DeepJIT: An end-to-end deep learning framework for just-in-time defect prediction,” in *Proc. 16th International Conference on Mining Software Repositories (MSR)*, 2019.
- [18] T. Hoang, H. J. Kang, D. Lo, and J. Lawall, “CC2Vec: Distributed representations of code changes,” in *Proc. 42nd International Conference on Software Engineering (ICSE)*, 2020.
- [19] Z. Zeng, Y. Zhang, H. Zhang, and L. Zhang, “Deep just-in-time defect prediction: How far are we?” in *Proc. 30th ACM SIGSOFT International Symposium on Software Testing and Analysis*

- (ISSTA), 2021.
- [20] C. Pornprasit and C. Tantithamthavorn, “JITLine: A simpler, better, faster, finer-grained just-in-time defect prediction,” in *Proc. 18th International Conference on Mining Software Repositories (MSR)*, 2021, pp. 369–379.
  - [21] N. Natarajan, I. S. Dhillon, P. Ravikumar, and A. Tewari, “Learning with noisy labels,” in *Advances in Neural Information Processing Systems (NIPS)*, 2013.
  - [22] B. Han, Q. Yao, X. Yu, G. Niu, M. Xu, W. Hu, I. Tsang, and M. Sugiyama, “Co-teaching: Robust training of deep neural networks with extremely noisy labels,” in *Advances in Neural Information Processing Systems (NeurIPS)*, 2018.
  - [23] C. G. Northcutt, L. Jiang, and I. L. Chuang, “Confident learning: Estimating uncertainty in dataset labels,” *Journal of Artificial Intelligence Research*, vol. 70, pp. 1373–1411, 2021.
  - [24] C. Ni, W. Wang, K. Yang, H. Gall, and X. Liu, “The best of both worlds: Integrating semantic features with expert features for defect prediction and localization,” in *Proc. 30th ACM Joint European Software Engineering Conference and Symposium on the Foundations of Software Engineering (ESEC/FSE)*, 2022.
  - [25] H. Keshavarz and M. Nagappan, “ApacheJIT: A large dataset for just-in-time defect prediction,” in *Proc. 19th International Conference on Mining Software Repositories (MSR)*, 2022.
  - [26] P. Mahbub, O. Shuvo, and M. M. Rahman, “Defectors: A large, diverse Python dataset for defect prediction,” in *Proc. 20th International Conference on Mining Software Repositories (MSR)*, 2023.
  - [27] B. W. Matthews, “Comparison of the predicted and observed secondary structure of T4 phage lysozyme,” *Biochimica et Biophysica Acta — Protein Structure*, vol. 405, no. 2, pp. 442–451, 1975.
  - [28] J. Cohen, “A coefficient of agreement for nominal scales,” *Educational and Psychological Measurement*, vol. 20, no. 1, pp. 37–46, 1960.
  - [29] F. Wilcoxon, “Individual comparisons by ranking methods,” *Biometrics Bulletin*, vol. 1, no. 6, pp. 80–83, 1945.
  - [30] J. L. Hodges and E. L. Lehmann, “Estimates of location based on rank tests,” *The Annals of Mathematical Statistics*, vol. 34, no. 2, pp. 598–611, 1963.
  - [31] S. Holm, “A simple sequentially rejective multiple test procedure,” *Scandinavian Journal of Statistics*, vol. 6, no. 2, pp. 65–70, 1979.